

LAB ACTIVITY (Exp-09)

Q1. Take random inputs from user and build a Max heap.

Q2. Implement the concept of Heap sort algorithm.

```
#include <stdio.h>
#include <stdlib.h>

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

void heapify(int arr[], int heapSize, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heapSize && arr[left] > arr[largest])
        largest = left;
    if (right < heapSize && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, heapSize, largest);
    }
}

void buildMaxHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

void heapSort(int arr[], int n) {
    buildMaxHeap(arr, n);
    for (int i = n-1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
int *arr = malloc(n * sizeof(int));

printf("Enter %d random numbers:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

buildMaxHeap(arr, n);
printf("\nArray after building Max-Heap:\n");
printArray(arr, n);

heapSort(arr, n);
printf("\nArray after Heap Sort:\n");
printArray(arr, n);

free(arr);
return 0;
}
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm> cd "d:\SEMESTER I\LAB
max_heap.c -o max_heap } ; if ($?) { .\max_heap }
Enter number of elements: 7
Enter 7 random numbers:
1 4 8 6 3 2 9

Array after building Max-Heap:
9 6 8 4 3 2 1

Array after Heap Sort:
1 2 3 4 6 8 9
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab10(1911)> |
```